

Desenvolvimento Back-End



Aula 02

- Tipos de dados e variáveis;
- Entrada e saída;
- Conversões de tipos;
- Pacotes;
- Encapsulamento;
- Modificadores de Visibilidade;

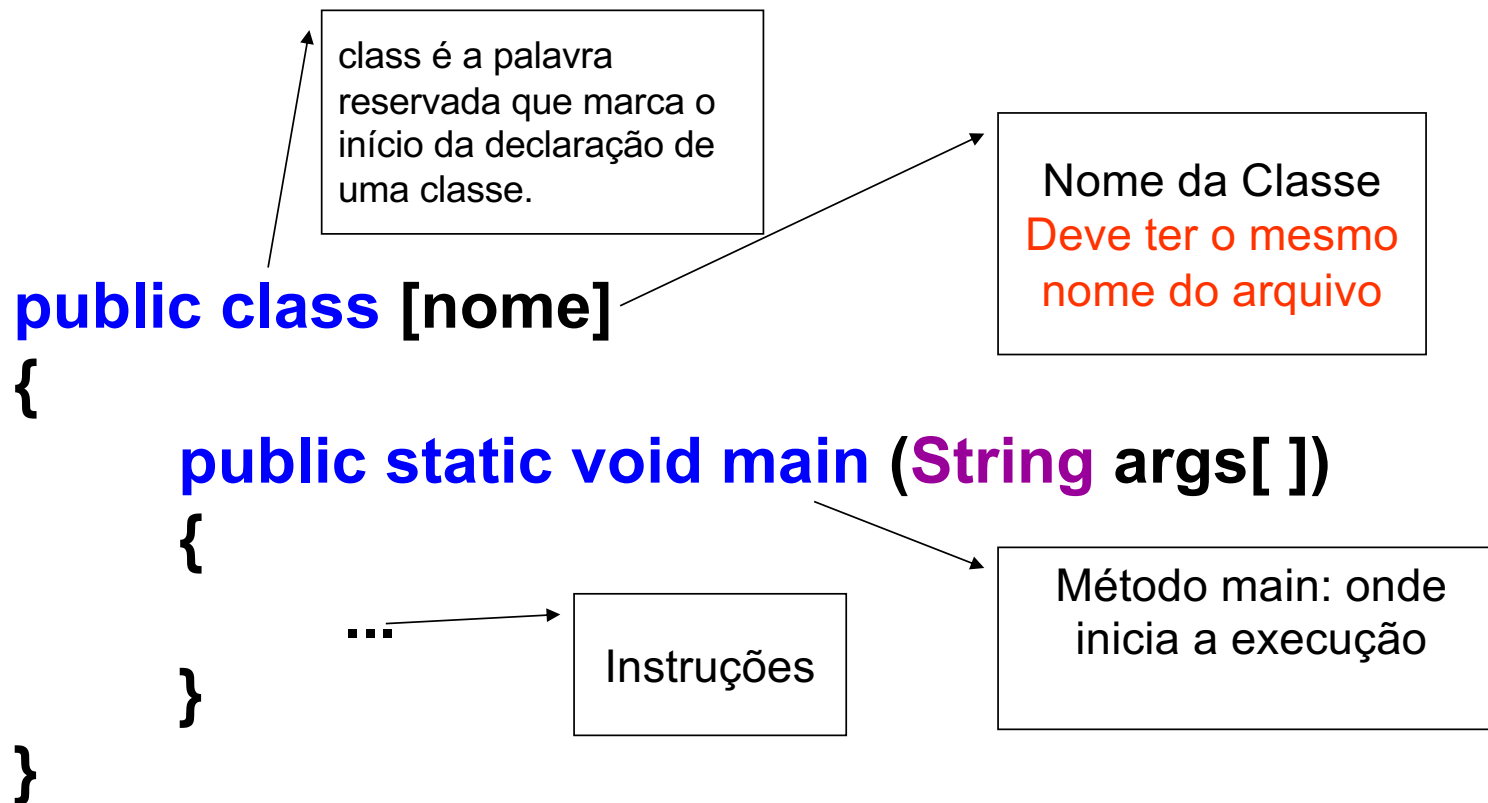
Exercício 01

Crie um programa em Java composto por duas classes. A primeira classe deve se chamar Carro e conter três atributos: modelo, ano e preço. Além disso, a classe deve possuir dois métodos: um método chamado aplicarDesconto, que recebe um valor percentual e retorna o preço do carro com o desconto aplicado, e um método chamado exibirInformacoes, que imprime no console o modelo, o ano e o preço do carro. A segunda classe deve se chamar UsaClasses e conter o método principal (main). Dentro desse método, crie três objetos da classe Carro, atribuindo valores fixos aos seus atributos. Em seguida, utilize o método aplicarDesconto para simular uma promoção em pelo menos um dos carros. Por fim, exiba no console as informações de todos os carros chamando o método exibirInformacoes.

Exercício 02

Crie um programa em Java composto por duas classes. A primeira classe deve se chamar Aluno e conter cinco atributos: nome, curso, ano de ingresso, número de disciplinas cursadas e média geral. Além disso, a classe deve possuir dois métodos: o primeiro deve se chamar calcularSemestresConcluidos e retornar quantos semestres o aluno já cursou, considerando que cada ano possui dois semestres e que o ano atual é 2025; o segundo deve se chamar aprovado e retornar verdadeiro se a média geral for maior ou igual a 6.0, e falso caso contrário. A segunda classe deve se chamar UsaClasses e conter o método principal (main). Dentro desse método, crie três objetos da classe Aluno, atribuindo valores fixos aos seus atributos. Em seguida, utilize os métodos criados para mostrar no console, junto das informações de cada aluno, quantos semestres ele já concluiu e se está aprovado ou não.

Estrutura de uma classe Java executável



Essa estrutura estará em todos os programas desenvolvidos em Java

Exemplo

Nome do
arquivo

```
Exemplo1.java
1  /*+++++++
2  Programa: Exibe mensagem na tela
3
4
5  ++++++
6
7  public class Exemplo1{
8      //o método main inicia a execução do aplicativo Java
9      public static void main (String args[ ]){
10         System.out.println("Primeiro exemplo");
11     } //fim do método main
12 } //fim da classe Exemplo1
```

O mesmo nome
do arquivo

/* Esse é um
comentário
com várias linhas*/

Inicia-se com o delimitador /* e
termina com */. O comentário
é ignorado pelo compilador.

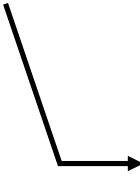
Dicas

Termine (quase...) todas as linhas com ;

Quando ocorrer um erro de compilação, dê um duplo clique sobre a mensagem de erro para destacar o comando errado no programa e verifique também a linha anterior, que pode ser a responsável pelo erro, especialmente se faltar o ;

Como convenção, todos os nomes de classe (identificador) em Java iniciam com uma letra maiúscula.

Use comentários, iniciados por // ou /* ... */



letras, dígitos, sublinhados(_) e sinais de cifrão (\$) que não iniciem com dígito e não contenham espaços em branco nem ponto. Em Java o nome Exemplo1 é diferente de exemplo1.

Tipos de Dados

- Em Java, os tipos são oito: **byte, short, int, long, float, double, char** e **boolean**.
- Esses tipos são portáveis entre todas as plataformas.
- Podem ser agrupados em quatro categorias:

Inteiros	Reais	Literal	Lógico
byte	float	char	boolean
short	double		
int			
long			

Tipos de dados

Tipo de dado	Tamanho em bytes e comentários
boolean	1 (valores true ou false)
char	2 (valores de 0 a 65535, conjunto de caracteres Unicode ISO)
byte	1 (valores de -128 a +127)
short	2 (valores inteiros de -32768 a 32767)
int	4 (valores inteiros de -2.xx bilhões a 2.xx bilhões)
long	8 (valores inteiros neg. e positivos de 19 dígitos)
float	4 (valores com casas decimais entre $\approx -3 \times 10^{38}$ e $+3 \times 10^{38}$)
double	8 (valores com casas decimais entre $\approx -1 \times 10^{308}$ e $+1 \times 10^{308}$)

String (conjunto de caracteres)

- Armazena um conjunto de dados alfanuméricos.
- Ele é sempre representado por aspas duplas.
Exemplos:
 - “Hoje”
 - “Aula de Java”
 - “Amanhã é sexta-feira, dia de *happy hour*.”

Variáveis

- São espaços ou alocações nas quais os dados são armazenados.
- A declaração de uma variável instrui o programa a reservar um espaço na memória, para que seja possível armazenar um dado de determinado tipo.
- A quantidade de memória reservada para uma variável é definida pelo seu tipo.

int $\Rightarrow$ 4 bytes (32 bits)

Declaração de Variáveis

Pode ser feita
em qualquer
parte do **corpo**
de uma classe.

Pode ser
composta por:

- Um tipo
- Um identificador (nome)
- Um valor (opcional)
- Ponto-e-vírgula

Exemplos de Declaração de Variáveis

```
//Tipos inteiros  
byte idade;  
short nro;  
int cod;  
long qtde;
```

```
//Tipos reais  
float preco;  
double receber;
```

```
//Tipo caracter  
char letra;
```

```
//Tipo lógico  
boolean escolha;
```

Identificador (nome):

- É através dele que você irá se referir a variável no código para lhe atribuir algum dado ou para recuperar um dado que foi armazenado.
- Regras para os nomes das variáveis:
 - Podem começar com letra minúscula ou maiúscula, underscore (_), ou símbolo de dólar (\$);
 - Não podem conter pontuação nem espaços;
 - Não podem ser utilizadas palavras reservadas.

Palavras reservadas

abstract	continue	goto	package	synchronized
assert	default	if	private	this
boolean	do	implements	protected	throw
break	double	import	public	throws
byte	else	instanceof	return	transient
case	extends	int	short	try
catch	final	interface	static	void
char	finally	long	strictfp	volatile
class	float	native	super	while
const	for	new	switch	

**As variáveis são
sensitive
Ar ≠ ar**

Nomes válidos:	Nomes inválidos:
codigo	1teste
quantidade	nome cliente
\$nome	cli.nome
_nro	x - y
indice1	código

Atribuição de valores a variáveis

- Existem várias maneiras de atribuir valores a variáveis:
 - Dizendo no código qual o valor a variável deve assumir:
 - Definir que uma variável assuma o valor de uma outra variável:

```
int cod = 123;  
float preco = 12.35F;  
char letra = 'A';
```

Identifica que o
valor é do tipo
float

```
int cod1, cod2;  
cod1 = 54;  
cod2 = cod1;
```

Atribuição de valores a variáveis

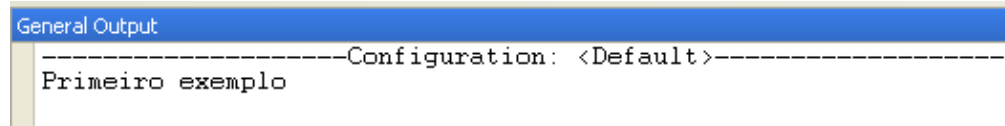
- Atribuir uma variável o resultado de uma expressão:

```
float = preco, qtde_comprada;  
double = receber;  
receber = preco*qtde_comprada;
```

Comandos de entrada e saída

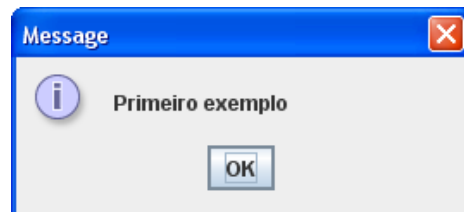
- Saída:
 - Modo texto:

```
System.out.println("Primeiro exemplo");
```



- Modo Gráfico:

```
JOptionPane.showMessageDialog(null, "Primeiro exemplo");
```



Comandos de saída – Modo Gráfico

```
JOptionPane.showMessageDialog(null, "Primeiro exemplo");
```

- Já está pronta, porém precisamos indicar que iremos utilizar e para isso temos que *importar* um pacote.

```
import javax.swing.JOptionPane;
```

- Carrega a classe **JOptionPane** do pacote **javax.swing**
- Ajuda a definir interfaces gráficas com o usuário para seu aplicativo
- Sempre coloquem esta diretiva no início do programa.

Detalhes do comando

```
JOptionPane.showMessageDialog(null,  
"Primeiro: "+valor1+ " Segundo: "+valor2);
```

- Indica uma chamada para o método ***showMessageDialog*** da classe **JOptionPane**, exige dois argumentos, separados por vírgula.
- **null** → ajuda o Java a posicionar a caixa de diálogo (caixa de diálogo aparece no centro da tela).

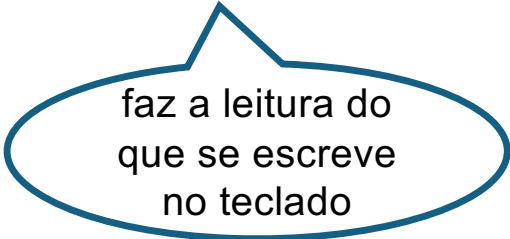
Entrada de Dados no Modo Texto

- Para utilizarmos o comando de entrada no modo texto, é necessário importar o seguinte pacote:

```
import java.util.Scanner;
```

- A classe Scanner ajuda na leitura dos dados informados. Para isso, é necessário criar um objeto do tipo Scanner que recebe o objeto System.in, do seguinte modo:

```
Scanner leia = new Scanner(System.in);
```



faz a leitura do
que se escreve
no teclado

Entrada de Dados no Modo Texto

- Para cada tipo de dado primitivo existe uma chamada do método para retornar o valor especificado na entrada de dados, sempre seguindo o formato `nextTipoDado()`.

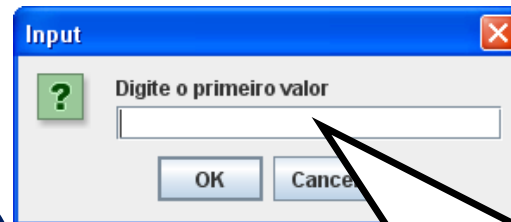
```
Scanner leia= new Scanner(System.in);  
float peso= leia.nextFloat();  
double salario = leia.nextDouble();  
int idade= leia.nextInt();  
byte valor1= leia.nextByte();  
long valor2= leia.nextLong();  
boolean b1 = leia.nextBoolean();  
String nome = leia.nextLine();
```

Comando de entrada – Modo Gráfico

- Modo Gráfico:

```
valor = JOptionPane.showInputDialog("Digite o primeiro valor");
```

A variável que
recebe conteúdo
é uma String

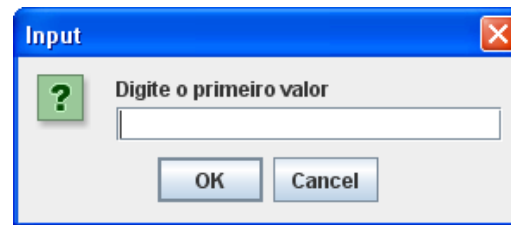


Mesmo que o usuário digite um
valor numérico, o conteúdo é
uma String

Também precisamos *importar* o pacote...

Comando de entrada – Modo Gráfico

```
valor = JOptionPane.showInputDialog("Digite o primeiro valor");
```



Podemos converter uma String para um dado do tipo numérico para realizar cálculos matemáticos, para isso:

```
n1 = Integer.parseInt(valor);
```

Declarada como int

Conversões de tipos – String para outros tipos

Para o tipo int:

- `Integer.parseInt (variável)`

Para o tipo double:

- `Double.parseDouble(variável)`

Para o tipo float:

- `Float.parseFloat(variável)`

Para o tipo long:

- `Integer.parseInt(variável)`

Para o tipo char:

- `entrada.charAt(0);`

Pacotes

Packages, ou **Pacotes**, são utilizados para **organizar** as Classes e as Bibliotecas de Classes em um Projeto.

Fisicamente um pacote é identificado com "um ou mais" **diretórios**.

Ex.: package **model**;

...é o diretório "**model**" que fica na raiz do Projeto Java

Ex.: package **com.mysql.jdbc**;

...é o diretório "**com/mysql/jdbc**" que fica na raiz do Projeto Java

Pacotes

Quando uma Classe está localizada em um **pacote**, nela precisa estar definida (em código) esta situação, identificando o pacote em que ela se localiza através da palavra reservada **package**.

```
package nome.do.pacote;  
//esta Classe pertence ao pacote "nome.do.pacote"
```

No entanto quando vamos utilizar (ou referenciar) ao longo da Classe, uma outra Classe que está em outro pacote, ou em outra Biblioteca, somos obrigados a importar a Classe, utilizando a palavra reservada **import**. E indicando o caminho **até a Classe**

```
import nome.do.pacote.Classe;
```

Caso se queira importar **todas as Classes** do pacote, no lugar no nome da Classe, colocamos ***** (**asterisco**).

```
import nome.do.pacote.*;
```

Modificadores de Visibilidade

public (público):

Uma declaração (**Classes, atributos ou métodos**) com o modificador *public* pode ser acessada de **qualquer** lugar e por qualquer entidade (Objeto) que possa visualizar a Classe a que este elemento pertence.

private (privado):

Uma declaração (**atributos ou métodos**) com o modificador *private* não pode ser acessado ou usado por nenhuma outra Classe , mas apenas por **métodos da própria Classe**. Esses atributos e métodos também não podem ser diretamente visualizados pelas Classes herdadas.

Modificadores de Visibilidade

protected (protegido):

O modificador *protected* torna o elemento (**atributos ou métodos**) acessível às **Classes do mesmo pacote ou através de herança**. Seus elementos não são acessíveis a outras classes fora do pacote em que foram declarados.

default (padrão):

Uma declaração (**Classes, atributos ou métodos**) em que não há definição de modificador, possui o que chamamos de modificador padrão (*default*), fazendo com que sejam acessíveis somente por **Classes do mesmo pacote**.

Modificadores de Visibilidade

Modificador	Classe	Pacote	Subclasse	Outros
public	Sim	Sim	Sim	Sim
protected	Sim	Sim	Sim	Não
default	Sim	Sim	Não	Não
private	Sim	Não	Não	Não

Encapsulamento

- É um importante conceito da orientação a objetos;
- Significado:
“princípio de design de código, ligado à programação orientada a objetos, que visa **proteger** os dados de um objeto contra modificações impróprias”
- Mas proteger de que ou de quem? De hackers?
- Não, o encapsulamento serve para manter as partes de um objeto juntos e permitir apenas que o próprio objeto seja responsável por seus atributos

Encapsulamento

- Encapsulamento (algumas vezes chamado de ocultamento de dados) é um conceito importante para o trabalho com objetos. Trata-se de reunir “encapsular” as propriedades e os métodos de uma determinada classe ou objeto, tornando o desenvolvimento e a manutenção do software mais simples, protegendo os dados de modificações e escondendo os detalhes internos da implementação. Cada objeto deve controlar e manipular o seu próprio estado

Encapsulamento

Ao contrário do que ocorre na programação mais tradicional, a orientação a objetos possui algumas características especiais para a utilização dos métodos e atributos. A única forma de conhecer ou alterar os atributos de um objeto é por meio dos seus métodos. A seguir algumas vantagens de utilizar o encapsulamento:

Encapsulamento

- O objeto é disponibilizado com toda sua funcionalidade, sem a necessidade de conhecimento de seu funcionamento ou armazenamento interno.
- É possível alterar um objeto internamente, adicionando métodos, sem que isso afete outros componentes do sistema que utilizam o objeto modificado.
- O processo de desenvolvimento de software é acelerado e simplificado, já que usuários dos objetos não precisam necessariamente saber como eles são constituídos internamente.
- A implementação de um comportamento pode ser modificada radicalmente sem que haja impacto no resto do programa. Isso é possível porque o código que utiliza o objeto não depende da forma como ele é implementado.

Encapsulamento

O encapsulamento é um dos principais conceitos da orientação a objetos e seu entendimento é simples: trata-se de combinar propriedades e métodos em um pacote e ocultar a sua implementação (propriedades e métodos). Não importa como a classe faz o trabalho, mas sim o que ela faz. Utilizar a visibilidade das propriedades e dos métodos é uma das maneiras de ocultar ou deixar acessíveis informações.

Encapsulamento

Pense em seu celular:

- Você o usa perfeitamente sem precisar entender de toda lógica complexa de como ele realmente funciona!
- Como a tela identifica o toque, e reage, como a memória é acionada e utilizada, como a câmera captura a luz e a transforma em arquivos etc.
- Isso é o encapsulamento: Esconder a lógica do mundo externo
- Em orientação a objetos obtemos isso dando responsabilidade certa aos objetos certos
- E alterações nas classes não podem refletir em efeitos no mundo externo (quem as utiliza)
- Como regra geral do encapsulamento: exponha apenas a quantidade de propriedades e métodos necessários da classe, para uso normal

Já sei tudo?

- Só isso? Já sei tudo que diz respeito à orientação a objetos?
- Não, ainda há um longo caminho pela frente, com muitos conceitos importantes ;
 - Abstração, Herança, Polimorfismo etc.
- Agora que tudo começa a ficar realmente divertido! 😬 😊 (é sério!)





That's all Folks!